Data Assimilation in Fluid Dynamics

1. Introduction and Motivation

Definition:

Set of techniques that combine experimental observations with mathematical models (based on physical laws) to estimate the actual state of a dynamic system more accurately.

History:

- Weather prediction (Richardson, 1922) to fluid dynamics
- Sasaki 1958 (Variational), Gandin 1963 (optimal interpolation)
- Supercomputing (70-80s): global forecasting
- 1990-2000s: Stochastic methods (Kalman filter)

Data Assimilation and Related Disciplines in Smart Sensing

Discipline	Relevance to Data Assimilation (DA)
Sparse reconstruction	Reconstructs flow fields from limited data; supports DA with incomplete or noisy data
Design of Experiments (DoE)	Determines optimal sensor placement for effective assimilation
Machine Learning (ML)	Complements DA in hybrid modeling or when physics-based models are incomplete
Spectral/modal decomposition (e.g., POD)	Reduces system dimensionality; enables efficient DA

Cross-Disciplinary Applications

- Meteorology, Oceanography: Weather, Ocean currents prediction
- Aerodynamics, Fluid Mechanics: Flow reconstruction
- Health sector (Bertoglio, 2012): Aortic wall stiffness (non-invasive)
- Natural prevention (Rochoux, 2020): Evolution of Wildfire Spread

Key components

- **Model (M), Observations (y)**: Predicts the system state and provides noisy real-world measurements.
- **Observation operator (H)**: Maps model state to observable quantities (e.g., from full state to measured variable).
- **Error covariances (B, R)**: Define uncertainty levels of model (B) and observations (R)—they control the weighting in the update.
- **Update Methods**: *Filtering* estimate present state taking into account the present time and previous observations until the present time, establish the most probable state; *Smoothing* adjusts the trajectory over a time window using past, present, and future observations.

Principal methods

- **Nudging**: Simple feedback correction based on observation mismatch.
- **Kalman Filter (KF)**: Optimal linear estimator with full error covariance propagation.
- **Ensemble Kalman Filter (EnKF)**: Uses ensembles to handle nonlinear, high-dimensional systems.
- **3D-Var**: Minimizes a cost function at a single time to blend model and observations.
- **4D-Var**: Optimizes the initial state using observations over a time window and model dynamics.

2.1. Nudging

Characteristics

- Nudging, also known as *Newtonian relaxation*, is one of the earliest and simplest data assimilation techniques.
- A simple data assimilation technique based on feedback.
- Corrects the model state by nudging it toward the observations.
- Adds a correction term proportional to the error between model and data.

$$\frac{dx}{dt} = f(x) + \alpha\,(y - H(x))$$

Here:

$x \rightarrow$ system state
$f(x) \rightarrow$ model evolution
$y \rightarrow$ observation
$H(x) \rightarrow$ maps model to observation space
$\alpha \rightarrow$ nudging (relaxation) coefficient

Advantages:

- Simple to implement and computationally efficient.
- Useful for stabilizing or guiding model trajectories.

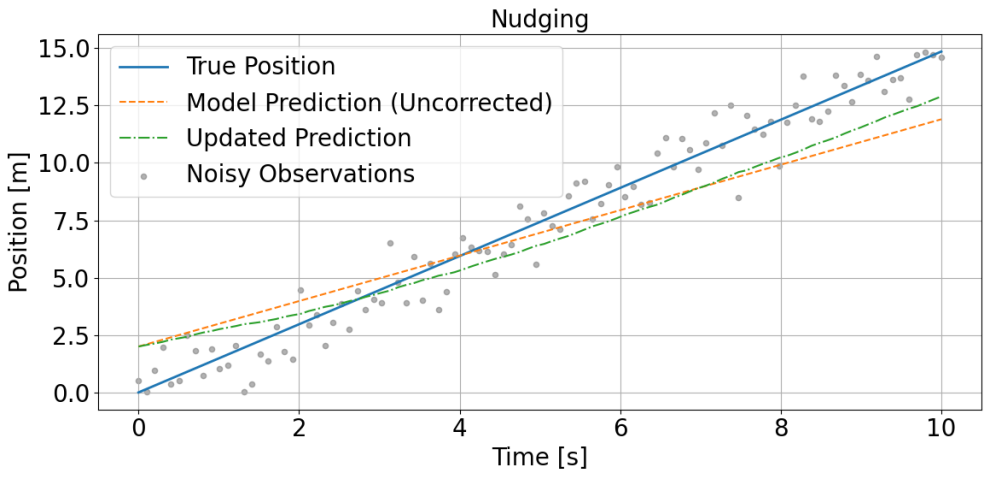
Limitations:

- No uncertainty quantification.
- Sensitive to the choice of α.
- Ignores spatial correlations in errors.

2.1 Nudging: Example

```
v_true = 1.5

for k in range(1, steps):
    x_model[k, 0] = x_model[k-1, 0] + dt * x_model[k-1, 1]
    x_model[k, 1] = x_model[k-1, 1]

alpha = 0.2

x_adjusted[0] = x_model[0]

for k in range(1, steps):
    dx = x_adjusted[k-1, 1] + alpha * (y_obs[k-1] - x_adjusted[k-1, 0])
    x_adjusted[k, 0] = x_adjusted[k-1, 0] + dt * dx
    x_adjusted[k, 1] = x_adjusted[k-1, 1]
```

2.1. Nudging: Example



Characteristics

The Kalman Filter (KF) is a recursive data assimilation algorithm that provides the optimal estimate of the state of a linear dynamical system with Gaussian errors. Update both the state estimate and its associated uncertainty as new observations become available.

- Recursive algorithm for estimating the state of a linear dynamic system.
- Combines predictions from a model with noisy measurements.
- Provides statistically optimal estimates under Gaussian assumptions.

Model (how are the variables bonded)

$$x_k = Fx_{k-1} + w_{k-1}, \quad w_k \sim \mathcal{N}(0, Q)$$

$$y_k = Hx_k + v_k, \quad v_k \sim \mathcal{N}(0, R)$$

- x_k: system state
- y_k: observation
- F, H: model matrices
- Q, R: covariance matrices for process and observation noise

Kalman Algorithm

Prediction step:

$$\hat{x}_k^- = F\hat{x}_{k-1}$$

$$P_k^- = FP_{k-1}F^\top + Q$$

Update step:

$$K_k = P_k^- H^\top (HP_k^- H^\top + R)^{-1}$$

$$\hat{x}_k = \hat{x}_k^- + K_k(y_k - H\hat{x}_k^-)$$

$$P_k = (I - K_k H)P_k^-$$

$F \rightarrow$ State transition matrix
$H \rightarrow$ Observation matrix
$Q \rightarrow$ Process noise covariance
$R \rightarrow$ Measurement noise covariance
$x_k \rightarrow$ State at time k
$\hat{x}_k^- \rightarrow$ Predicted state
$\hat{x}_k \rightarrow$ Updated state
$P_k^- \rightarrow$ Predicted covariance
$P_k \rightarrow$ Updated covariance
$K_k \rightarrow$ Kalman Gain
$y_k \rightarrow$ Observation
$S \rightarrow$ Innovation covariance
$I \rightarrow$ Identity matrix

Advantages:

- Provides statistically optimal estimate under Gaussian linear assumptions.
- Updates uncertainty along with state.
- Recursive and efficient for low-dimensional systems.

Limitations:

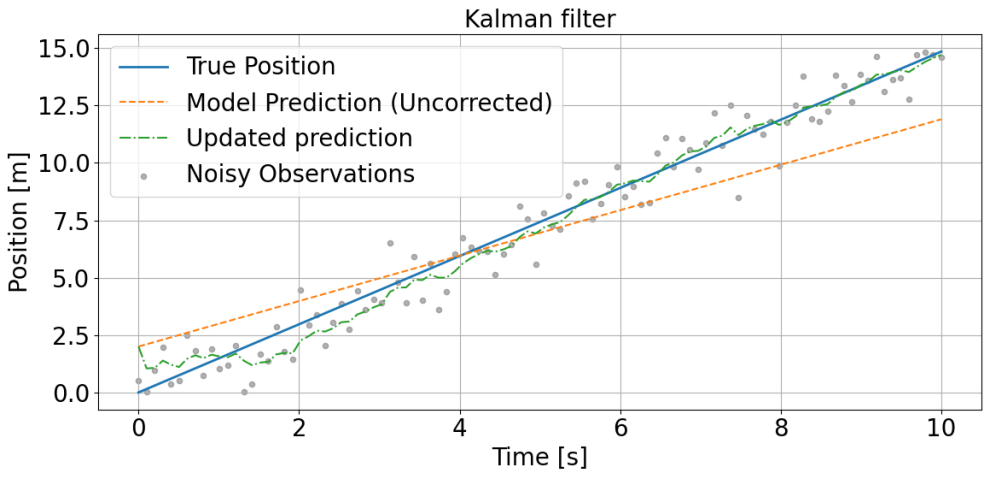
- Requires linear dynamics and observation models.
- Assumes Gaussian errors.
- Poor performance in nonlinear or high-dimensional systems.

2.2. Kalman Filter: Example

```
# Kalman Filter parameters
F = np.array([[1, dt], [0, 1]])
H = np.array([[1, 0]])
Q = np.array([[0.01, 0], [0, 0.01]])
# model noise
R = np.array([[pos_noise_std**2]])
# observation noise
I = np.eye(2)

# Initial estimates
x_kf = np.zeros((steps, 2))
x_kf[0] = [2.0, 1.0]
P = np.eye(2)
```

```
for k in range(1, steps):
    # Predict
    x_pred = F @ x_kf[k-1]
    P_pred = F @ P @ F.T + Q

    # Update
    y = y_obs[k] - H @ x_pred
    S = H @ P_pred @ H.T + R
    K = P_pred @ H.T @ np.linalg.inv(S)
    x_kf[k] = x_pred + K @ y
    P = (I - K @ H) @ P_pred
```

2.2. Kalman Filter: Example



Characteristics

The Ensemble Kalman Filter (EnKF) is an extension of the Kalman Filter designed to handle nonlinear and high-dimensional systems. Originally proposed by Geir Evensen (Evensen 1994), it replaces the explicit computation of error covariance matrices with Monte Carlo estimates using an ensemble of model realizations.

EnKF Algorithm

1. Initial state x_0 with an uncertainty P_0
2. Generate N members (ensamble): $x_0^{(i)} \sim \mathcal{N}(x_0, P_0) \rightarrow x_{k-1}^{a(i)}$
3. Transition: $x_k^{f(i)} = \mathcal{M}(x_{k-1}^{a(i)}) + w_k^{(i)}$
4. Update: $x_{k-1}^{a(i)} = x_{k-1}^{f(i)} + K_k(y_k^{(i)} - \mathcal{H} x_k^{f(i)})$

$$K_k = P_k^f \mathcal{H}^\top (\mathcal{H} P_k^f \mathcal{H}^\top + R)^{-1}$$

$$P_k^f = \frac{1}{N-1} \sum_{i=1}^{N} (x_k^{f(i)} - \bar{x}_k^f)(x_k^{f(i)} - \bar{x}_k^f)^T$$

$x_k^{f(i)} \rightarrow$ Forecast state (prior)
$x_k^{a(i)} \rightarrow$ Assimilated state (posterior)
$\bar{x}_k^f \rightarrow$ Ensemble mean forecast
$P_k^f \rightarrow$ Forecast covariance
$K_k \rightarrow$ Kalman Gain
$\mathcal{M}$, $\mathcal{H}$: Model and observation operators
$w_k^{(i)} \rightarrow$ Model noise
$y_k^{(i)} \rightarrow$ Perturbed observations
$N \rightarrow$ Number of ensemble members
$R \rightarrow$ Observation noise covariance

2.3. Ensemble Kalman Filter (EnKF)

Advantages:

- Suitable for nonlinear and high-dimensional systems.
- No need to compute or store full covariance matrices.
- Provides flow-dependent error covariances.

Limitations:

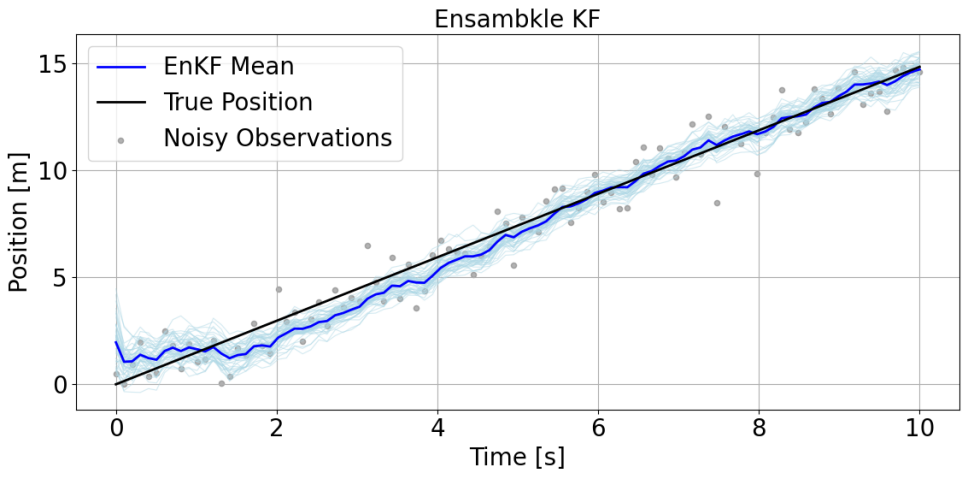
- Sampling errors for small ensembles.
- Requires ensemble inflation and localization to control spurious correlations.

2.3. Ensamble Kalman Filter: Example

```
# Parámetros del ensemble
N = 50
# número de miembros del ensemble

# Inicialización del ensemble
x_ens = np.zeros((steps, N, 2))
# [tiempo, miembro, estado]
x_ens[0, :, 0] = 2.0 + np.random.normal(0, 1.0, N)
# posición inicial
x_ens[0, :, 1] = 1.0 + np.random.normal(0, 0.5, N)
# velocidad inicial

# Matrices del modelo
F = np.array([[1, dt], [0, 1]])
H = np.array([[1, 0]])
R = np.array([[pos_noise_std**2]])

# Evolución del EnKF
x_enkf_mean = np.zeros((steps, 2))
# media del ensemble (estimación)
x_enkf_mean[0] = np.mean(x_ens[0], axis=0)
```

```
for k in range(1, steps):
    # Predicción de cada miembro
    for i in range(N):
        noise = np.random.multivariate_normal([0, 0], Q)
        x_ens[k, i] = F @ x_ens[k-1, i] + noise

    # Calcular estadísticos del ensemble
    x_mean = np.mean(x_ens[k], axis=0)
    X_prime = x_ens[k] - x_mean
# perturbaciones
    P_f = (X_prime.T @ X_prime) / (N - 1)

    # Perturbar observación
    y_perturbed = y_obs[k] + np.random.normal(0, pos_noise_std, N)

    # Actualización de cada miembro
    for i in range(N):
        Hx = H @ x_ens[k, i]
        S = H @ P_f @ H.T + R
        K = P_f @ H.T @ np.linalg.inv(S)
        innovation = y_perturbed[i] - Hx
        x_ens[k, i] += (K @ innovation).flatten()

    x_enkf_mean[k] = np.mean(x_ens[k], axis=0)
```

2.3. Ensamble Kalman Filter: Example



2.4. 3D-Var (Three-Dimensional Variational

3D-Var (Three-Dimensional Variational Data Assimilation) is a method used to optimally estimate the system state at a single point in time. It combines:

- A background forecast x^b (from the model),
- Noisy observations y,
- A cost function that penalizes deviations from both.

$$J(x) = \frac{1}{2}(x - x^b)^T B^{-1}(x - x^b) + \frac{1}{2}(y - Hx)^T R^{-1}(y - Hx)$$

Where:

x: the analysis (corrected state)

x^b: background state

y: observations

H: observation operator

B: background error covariance matrix

R: observation error covariance matrix

2.4. 3D-Var (Three-Dimensional Variational)

Interpretation:

- The first term measures the deviation from the model.
- The second term measures the deviation from the observations.
- The solution balances both, weighted by their respective uncertainties.

Solution: In the linear case, the minimizer has a closed-form expression:

$$x^a = x^b + K(y - Hx^b)$$

$$K = BH^T(HBH^T + R)^{-1}$$

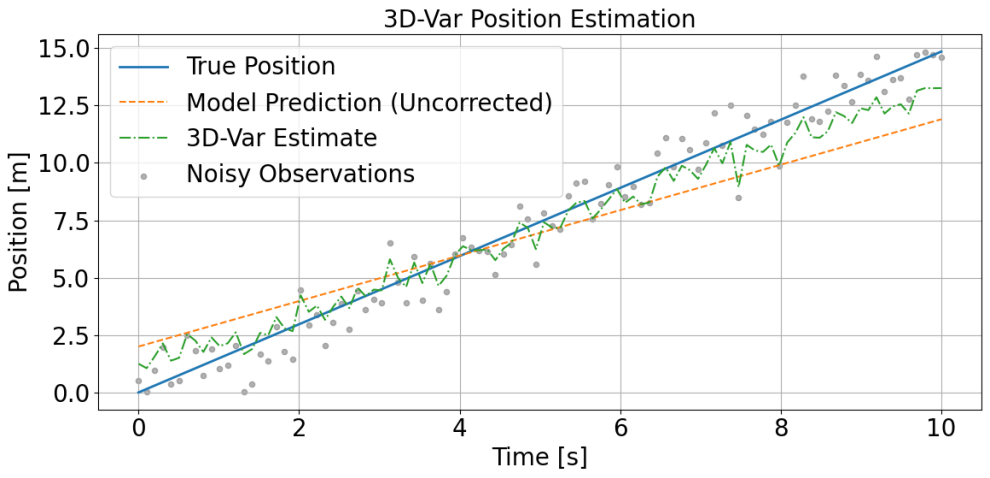
Summary:

- 3D-Var updates the state at a single time step.
- It uses statistical covariances to balance trust in model vs. observations.
- It does not use temporal dynamics (unlike 4D-Var).

2.4. 3D-Var (Three-Dimensional Variational): Example

```
# Supuestos iniciales
B = np.diag([1.0, 1.0])  # Background covariance
R = np.array([[pos_noise_std**2]])
# Observation error covariance
H = np.array([[1, 0]])  # Observa solo la posición

# Estado de fondo (background) sin corrección
x_b = x_model.copy()

# Matriz de ganancia variacional (constante en este caso lineal)
HBHT_R_inv = np.linalg.inv(H @ B @ H.T + R)
K_3dvar = B @ H.T @ HBHT_R_inv

# Corrección 3D-Var en cada instante (independiente)
x_3dvar = np.zeros_like(x_b)
for k in range(steps):
    innovation = y_obs[k] - (H @ x_b[k])
    x_3dvar[k] = x_b[k] + (K_3dvar @ innovation).flatten()
```

2.4. 3D-Var (Three-Dimensional Variational): Example



2.5. 4D-Var (Four-Dimensional Variational)

4D-Var (Four-Dimensional Variational Data Assimilation) estimates the optimal initial state of a system over a time window by minimizing the mismatch between model forecasts and observations.

- Incorporates temporal evolution of the model.
- Seeks the best initial state x_0 that explains all observations within a time window.
- Requires integration of the model forward in time.

$$J(x_0) = \frac{1}{2}(x_0 - x^b)^T B^{-1}(x_0 - x^b) + \frac{1}{2}\sum_{k=0}^{N}(y_k - H\mathscr{M}_k(x_0))^T R^{-1}(y_k - H\mathscr{M}_k(x_0))$$

Where:

x_0: initial condition to be optimized

x^b: background initial state

y_k: observations at time t_k

H: observation operator

B: background error covariance

R: observation error covariance

$\mathscr{M}_k$: model evolution operator to t_k

2.5. 4D-Var (Four-Dimensional Variational)

Interpretation:

- The first term penalizes deviation from the background.
- The second term penalizes misfit between model outputs and observations over time.
- The optimal state minimizes the combined penalty over the time window.

Features:

- Produces a globally consistent solution over time.
- Requires an adjoint or tangent-linear model for gradient computation.
- Computationally expensive but very accurate.

Comparison:

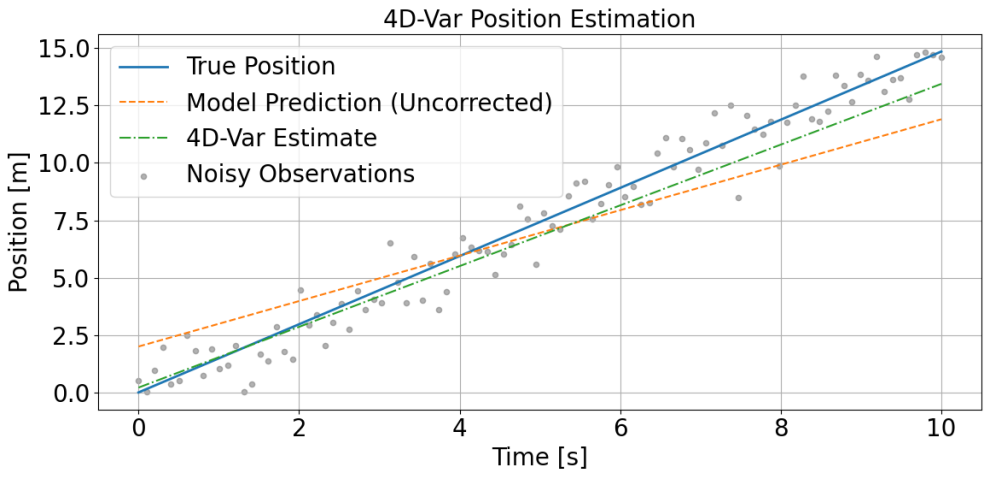
- Unlike 3D-Var, 4D-Var assimilates observations over time.
- It optimizes the initial state, not instantaneous values.

2.5. 4D-Var (Four-Dimensional Variational): Example

```
# Parámetros
B = np.diag([1.0, 1.0])
# Background covariance
R = np.array([[pos_noise_std**2]])
H = np.array([[1, 0]])

# Modelo lineal
def model_propagate(x0, steps, F):
    traj = np.zeros((steps, 2))
    traj[0] = x0
    for k in range(1, steps):
        traj[k] = F @ traj[k-1]
    return traj

# Cost function parametrizada por la ventana
def J(x0, window):
    x0 = np.array(x0)
    traj = model_propagate(x0, window, F)
    innov = y_obs[:window] - traj[:, 0]
    obs_term = np.sum((innov**2) / R[0, 0])
    back_term = (x0 - x_model[0]).T @ np.linalg.inv(B) @ (x0 - x_model[0])
    return 0.5 * (obs_term + back_term)
```

```
# Longitud de la ventana deseada
window = 50
# por ejemplo, 3 segundos si dt = 0.1

# Minimización con la ventana seleccionada
res = minimize(J, x0=np.array([2.0, 1.0]), args=(window,), method='L-BFGS-B')
x0_opt = res.x

# Evolución del estado óptimo hasta el final (o solo la ventana)
x_4dvar = model_propagate(x0_opt, steps, F)
```

2.5. 4D-Var (Four-Dimensional Variational): Example



Comparison of Data Assimilation Methods